Performance-Aspects and -Analysis for MPI & OpenMP applications

HLRS

Thirty Years, One Vision



Christoph Niethammer <niethammer@hlrs.de>

Amdahl's Law (strong scaling, fixed problem size)

The execution time $t_{parallel,\ p}$ of a parallel program running on **p** processors can be modeled from t_{serial} and a factor **f** of the **sequential part** of code that can not be done in parallel

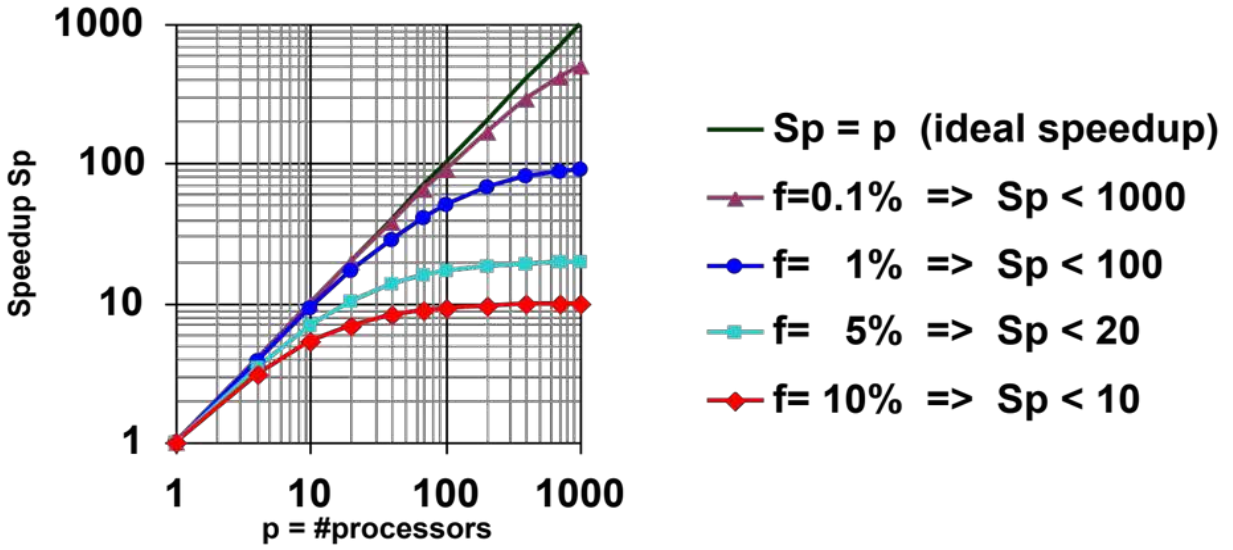
$$t_{parallel,\ p} = f \cdot t_{serial} + (1-f) \cdot t_{serial} / p + t_{communication} + t_{load_imbalance} + t_{serialization} + \dots - t_{super_scalar_effects}$$

Often the **Speedup S** is mentioned here:

$$S_p = t_{serial} / t_{parallel,\ p} = 1 / (f + (1-f) / p)$$

Speedup S is limited following Amdahl:

$$\lim_{p \to \infty} S_p = 1 / f$$



Optimmization beyond Amdahl ...

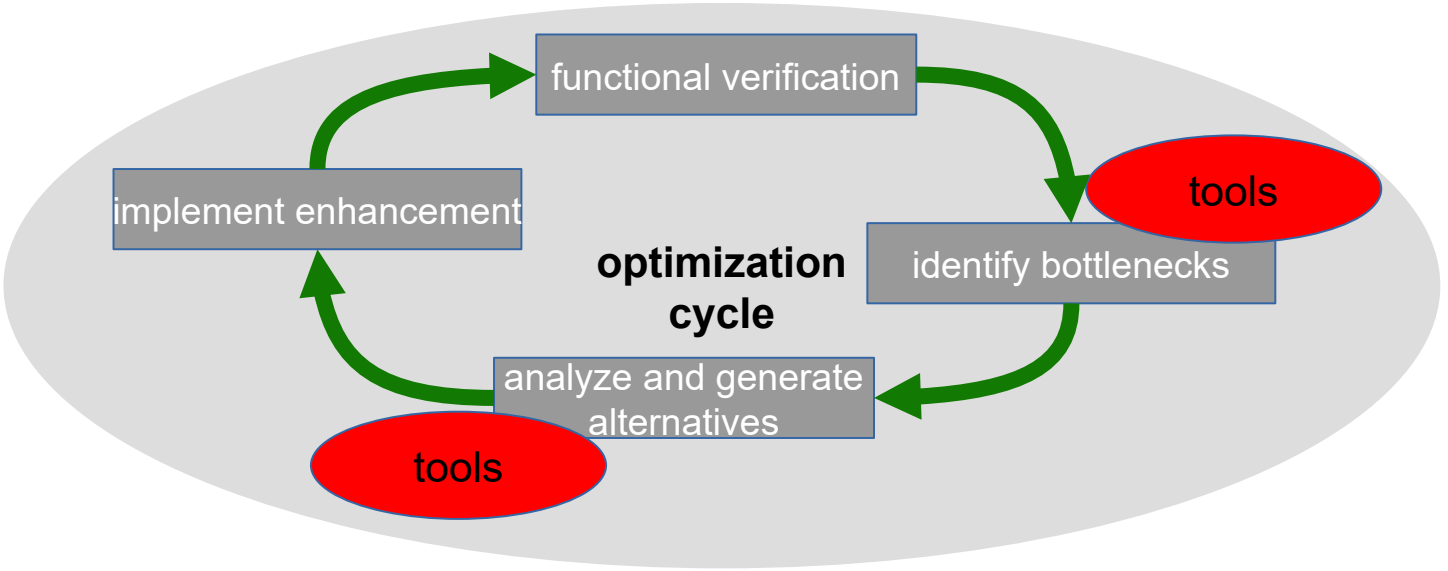
Once you have optimized your code and minimized the sequential parts (f <<< 1) the previously ignored parts get interesting:

$$t_{parallel,\ p} = f \cdot t_{serial} + (1-f) \cdot t_{serial} / p + t_{communication} + t_{load_imbalance} + t_{serialization} + \ldots - t_{super_scalar_effects}$$

Performance Optimization - Introduction

HLRS
Thirty Years, One Vision

- Code optimization requires a lot of effort and makes code often harder to maintain
- Therefore **apply optimizations only to code parts that matter**!
- Main task:
  identify critical parts of the progam
- Sometimes critical parts are not obvious for the programmer



Basics of Performance Analysis

Performance Metrics

HLRS
Thirty Years, One Vision

- **Time:**
  - How fast does my code run?
  - Which are the most time consuming parts?



- **Hardware / software counter**
  - Why does my program run so slow?
  - What kind of optimizations can be benefficial?
  - How far is my program away from the hardware performance?



Data Aquisition: Tracing vs. Profiling

H L R S
Thirty Years, One Vision

Tracing

- Record events in their timely order



event	start	end
A	0	3
B	3	5
C	5	6
B	6	7
...	...	...

Profiling

- Record statistical data about the occurence of events, e.g., total number of function calls

event	count	total time
A	1	3
B	2	3
C	1	1
...	...	...

Data Aquisition: Instrumentation vs. Sampling

HLRS
Thirty Years, One Vision

Instrumentation

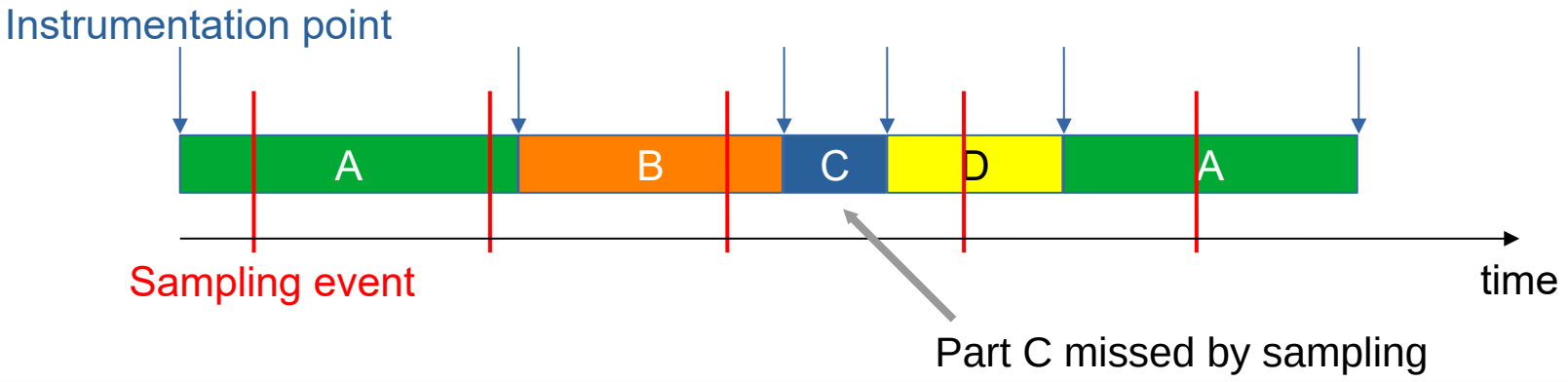
- Collects information at specific points in the code, e.g., function entry and exit
- Mostly used to get a timeline of events

- Can result in high overhead
- Produces typically lots of data

Sampling

- Timer/event triggered collection process
- Takes typically call stack information

- Low overhead
- Can miss necessary information



MPI's PMPI Interface

HLRS
Thirty Years, One Vision

The MPI standard comes with great support for tools :-)

The simplest and most widely used interface is the **PMPI** interface, which allows easy instrumentation: Each MPI function is a wrapper to a PMPI function:
This allows overloading of functions by tools via LD_PRELOAD, e.g., for time meassurments:

```
int MPI_Send(...) {
  double start, end;
  start = PMPI_Wtime();
  int ret = PMPI_Send(…); /* the original MPI operation */
  end = PMPI_Wtime();
  my_tool_record(start, end)
  return ret;
}
```

Common Performance Issues for parallel Programs

Common Performance Issues in HPC Applications

HLRS
Thirty Years, One Vision

- **Load imbalance**
- **Serialisation**
- **Communication overheads**
- **Runtime overheads**
- Memory limitation
  - Cacheline usage
  - Cache trashing
- Compute limitation
  - Vectorization
  - Port usage

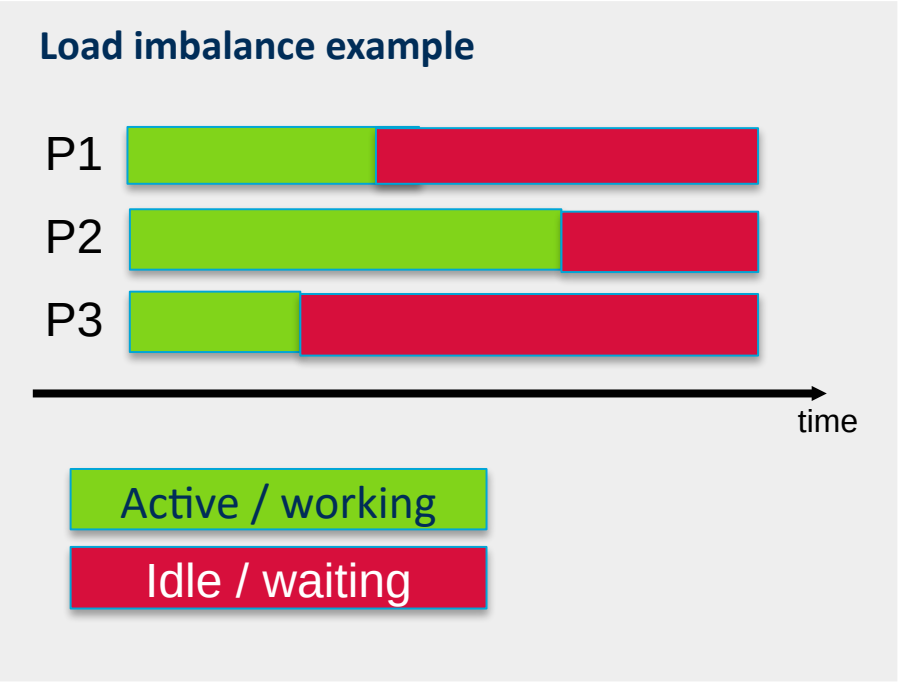
…

Load Imbalance

HLRS
Thirty Years, One Vision

Definition:

Parallel **workers perform different amounts of work or take different amounts of time**, leaving some workers idle while others are still executing.



Types of imbalance:

- **Instruction imbalance:**
  Different numbers of instructions executed (e.g., divergent control flow, different loop iteration counts).
- **Computational imbalance:**
  Similar instruction counts, but different execution times due to varying operation costs, data dependencies, cache misses, or memory stalls.

Impact: Reduced parallel efficiency; the slowest worker determines completion time.

Load Imbalance

HLRS
Thirty Years, One Vision

MPI-specific
- Uneven distribution of data or work across MPI processes
- Waiting at collective operations or point-to-point communication
- Static partitioning that does not reflect actual workload

OpenMP-specific
- Uneven loop iteration costs with schedule(static)
- Task workloads of varying duration
- Thread waiting at barriers for the slowest thread

Mitigation strategies:
- Better workload partitioning
- Dynamic scheduling or work stealing
- Reducing synchronization points (MPI-collectives / OpenMP-nowait)

Serialisation

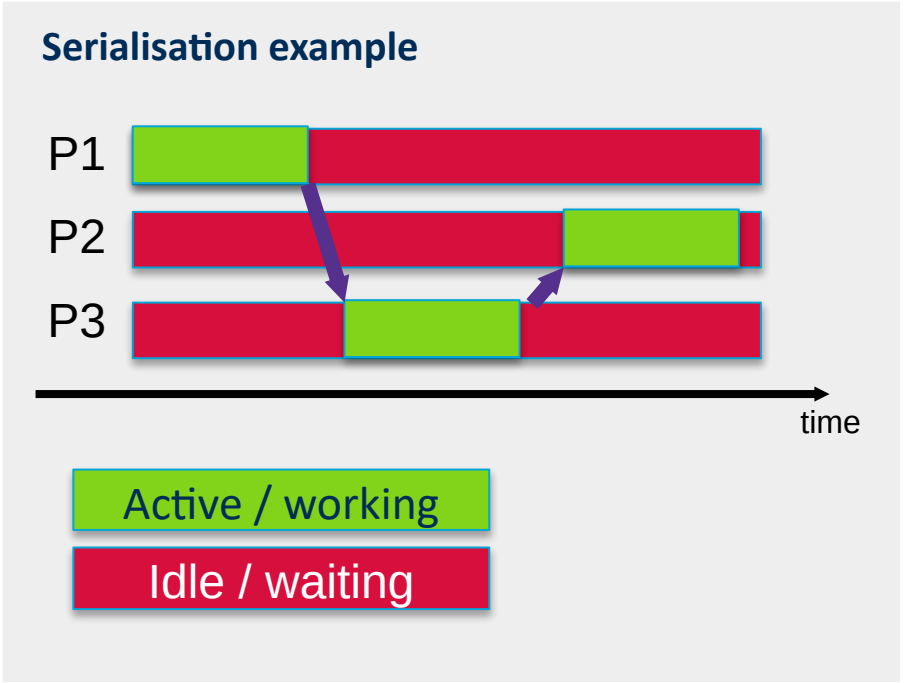
HLRS
Thirty Years, One Vision

Definition:

Parallel **workers are forced to execute work sequentially** because of dependencies, shared resources, or communication patterns that limit concurrent progress.

Common causes:

- **Data dependencies:** A computation must wait for results from another worker.
- **Resource contention:** Multiple workers compete for a shared resource. (cf. locks or critical sections restricing concurrent execution)
- **Centralized work:** A single worker performs tasks that could otherwise be distributed.



Impact: Limits parallel speedup; additional workers may provide little or no benefit.

Serialisation

MPI-specific

- **Communication dependencies**: Processes wait for messages before proceeding.
- **Sequential communication:** Blocking communication patterns prevent useful computation from overlapping with communication.

OpenMP-specific

- **Critical sections and locks:** Only one thread can execute a protected region at a time.

Mitigation strategies:

- Remove unnecessary dependencies
- Reduce shared updates; use local accumulation followed by reduction
- overlap communication with computation (→ nonblocking MPI operations)

Communication and Runtime Overheads

Definition:

Time spent exchanging data, coordinating parallel workers, and managing parallel execution rather than performing useful computation.

Communication overheads

- **Latency and bandwidth:**
  Cost of initiating messages and transferring data
  $t_{transfer}$ = latency + msg_size / bandwidth

- **Communication frequency and volume:**
  Many small messages or excessive data exchange.

Runtime overheads

- **Work management:**
  Thread/process management
- **Synchronization mechanisms:**
  Locks, atomics, …
- **Communication runtime:**
  Message queues & message matching, progress, and collective algorithms.

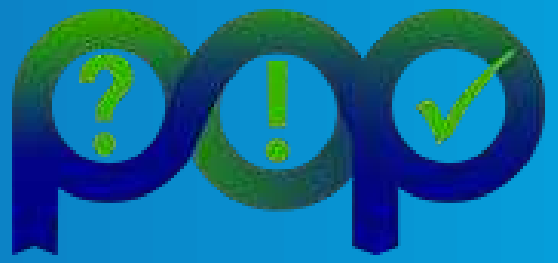
Impact: Reduces useful computation, increased waiting time

Communication and Runtime Overheads

MPI-specific
- Point-to-point communication (MPI_Send, MPI_Recv)
- Collective operations (MPI_Bcast, MPI_Reduce, MPI_Alltoall)
- Message matching, communication progress, and synchronization

OpenMP-specific
- Thread-team creation and management
- Loop scheduling (static, dynamic, guided)
- Task creation and scheduling
- Barriers, critical sections, and atomic operations

Mitigation strategies:
- Reduce communication frequency and volume
- Overlap communication with computation where possible
- Increase task granularity
- Reuse thread teams, choose appropriate scheduling



POP Standard Metrics for Parallel Performance Analysis

POP Metrics

HLRS
Thirty Years, One Vision

POP-Project (Performance Optimization and Productivity Center of Excellence in HPC)

Provides Optimization services to academic and industrial code(s) in all domains



POP Metrics:

- Quantify parallel inefficiencies and
- Identify their main causes using a hierarchy of efficiency metrics.

Core idea:

- Calculate efficiency metrics from execution profiles or traces.
- Higher efficiency generally indicates better performance; lower values indicate potential bottlenecks.
- Decompose overall efficiency into meaningful factors to guide optimization.

https://pop-coe.eu/

POP Top-level metrics

HLRS
Thirty Years, One Vision

- **Global Efficiency (GE):** Overall quality of parallelization and computational scaling.

- **Parallel Efficiency (PE):** Efficiency of the parallel execution itself.
- **Computation Scaling (CompS):** How the amount and speed of useful computation change relative to a reference run.

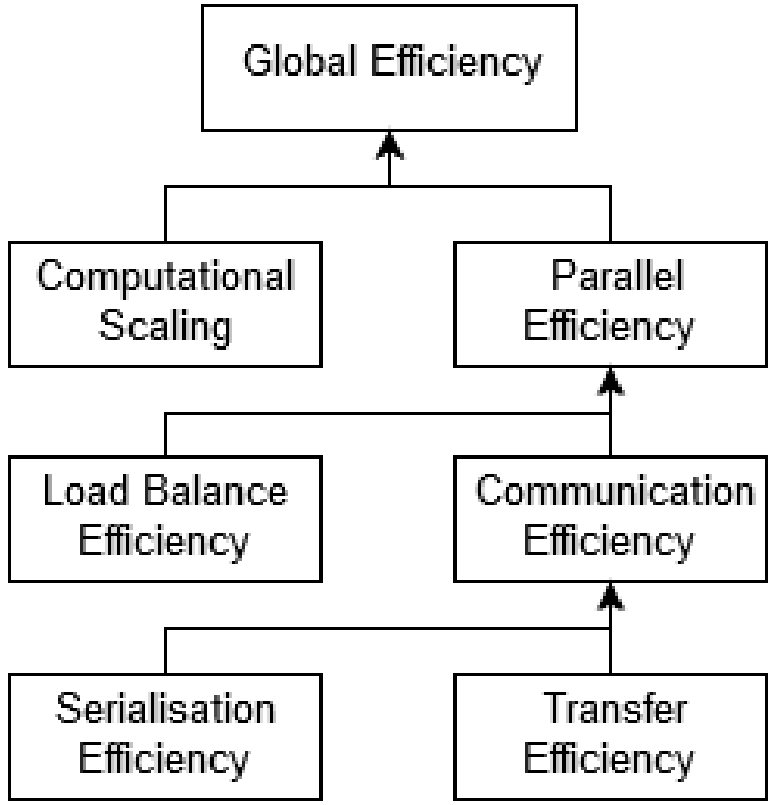
  `GE = PE × CompS`

- **Load Balance (LB):** ratio between average useful computation time and maximum useful computation time:

  `LB = avg useful comp. time / max comp. time`

- **Communication Efficiency (CommE):** ratio between maximum useful computation time and total runtime:

  `CommE = max comp. time / total runtime`



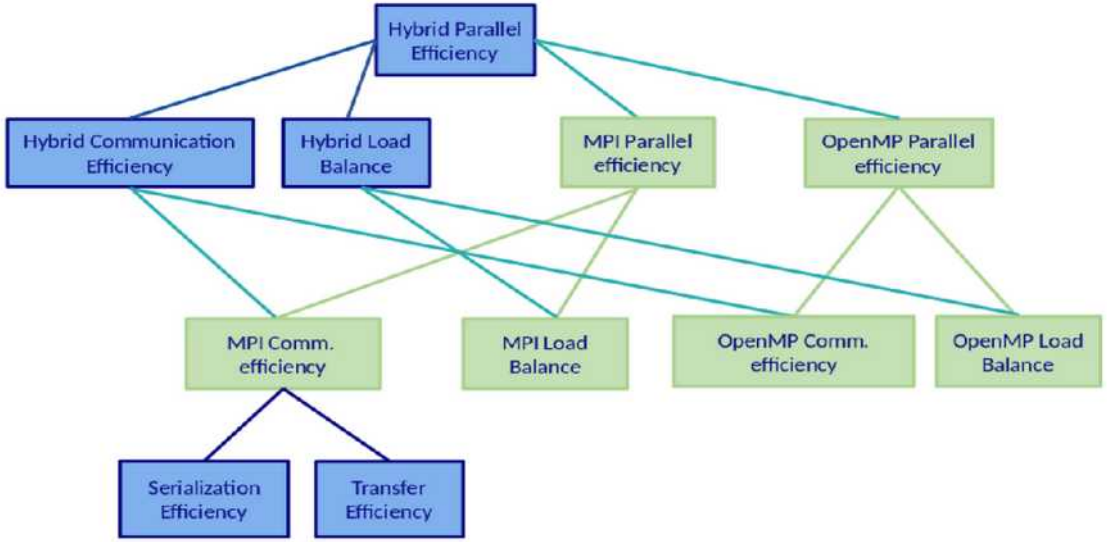
POP Metrics – Hybrid MPI + OpenMP

HLRS
Thirty Years, One Vision

Metrics for Hybrid MPI+OpenMP are not so easy.
Here POP provides two metric schemes:

Additive scheme: Expresses inefficiencies as additive contributions from process-level and thread-level parallelism. Useful for quantifying the absolute cost of individual bottlenecks.

Multiplicative scheme: Decomposes efficiency into products of sub-metrics, such as load balance and communication efficiency. Useful for a hierarchical, programming-model-independent view of parallel efficiency.



Scheme for multiplicative POP metrics

Common tools for performance analysis

Performance Tools – Overview

Tool	serial	parallel	HW-counter	GUI	free
gprof	x				x
perf	x		x		x
likwid	x		x		x
mpiP		x			x
Score-P + CUBE		x	x	x	x
Extrae+Paraver	x	x	x	x	x
Score-P + Vampir		x	x	x	

gprof

GPROF

HLRS
Thirty Years, One Vision

gprof can be used to collect

- runtime distribution (via sampling)
- call counts

of functions in a program

- collects Information at runtime
- requires the program to be build with specific compiler options
- uses post-run analysis of the data

Resources:

http://sourceware.org/binutils/docs/gprof/

gprof: a Call Graph Execution Profiler, Susan L. Graham Peter B. Kessler Marshall K. McKusick, 1982

GPROF – Usage

The program must be compiled and linked with the -pg option:
gcc -pg -O3 myapp.c -o myapp

Run the application
./myapp

Will produce a gmon.out file which can be analysed with
gprof myapp

GPROF – Profiling MPI Applications

HLRS
Thirty Years, One Vision

To profile MPI applications with gprof set the environment variable

```
GMON_OUT_PREFIX=gmon.out
```

which will create one file per process with name

```
gmon.out.[PID]
```

Merge output files with

```
gprof -s <binary> gmon.out.*
```

If you want to keep MPI rank information use a wrapper script as shown

```
mpirun -np <np> ./wrapper.sh <app>
```

(works only using glibc)

wrapper.sh

```
#!/bin/bash
PDIR=profdir.$PMI_RANK
export GMON_OUT_PREFIX=$
   {GMON_OUT_PREFIX:-gmon.out.mpi}.
   $PMI_RANK
exec $@
```

GPROF Output – Flat Profile

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
33.23	1.12	1.12	15756	0.07	0.07	void VectorizedCellProcessor::_calculatePairs<CellPairPolicy_<true>, true, MaskingChooser>(CellDataSoA&, CellDataSoA&)
24.04	1.93	0.81	14948	0.05	0.05	void VectorizedCellProcessor::_calculatePairs<CellPairPolicy_<true>, false, MaskingChooser>(CellDataSoA&, CellDataSoA&)
6.53	2.15	0.22	1212	0.18	0.18	void VectorizedCellProcessor::_calculatePairs<SingleCellPolicy_<true>, true, MaskingChooser>(CellDataSoA&, CellDataSoA&)
4.75	2.31	0.16	18180	0.01	0.02	FullParticleCell::buildSoACaches()
4.60	2.47	0.16	1444489	0.00	0.00	ParticleCellBase::testPointInCell(double const*) const
2.97	2.57	0.10	1444489	0.00	0.00	LinkedCells::getCellIndexOfPoint(double const*) const
2.37	2.65	0.08	1639530	0.00	0.00	FullMolecule::setupSoACache(CellDataSoABase*, unsigned int, unsigned int, unsigned int, unsigned int)
2.37	2.73	0.08	814416	0.00	0.00	CommunicationBuffer::addHaloMolecule(unsigned long, FullMolecule const&)
2.37	2.81	0.08	4944112	0.00	0.00	FullMolecule::FullMolecule(FullMolecule const&)
2.08	2.88	0.07	1639530	0.00	0.00	FullMolecule::calcFM()
1.48	2.93	0.05	3279060	0.00	0.00	Quaternion::rotate(std::array<double, 3ul> const&) const
1.48	2.98	0.05	34452	0.00	0.00	FullParticleCell::deallocateAllParticles()

GPROF Output – Call Graph


```
index % time    self  children    called     name
-----------------------------------------------
                0.00    0.02       4/404        Simulation::prepare_start() [29]
                0.00    2.13     400/404        Simulation::simulate() [1]
[2]     63.8    0.00    2.15     404          LinkedCells::traverseCells(CellProcessor&) [2]
                0.00    2.15     404/404        SlicedCellPairTraversal<FullParticleCell>::traverseCellPairs(CellProcessor&) [4]
                0.00    0.00       4/19         LinkedCells::getNumberOfParticles() [89]
                0.00    0.00     404/404        VectorizedCellProcessor::initTraversal() [202]
                0.00    0.00       4/4          TraversalTuner<FullParticleCell>::findOptimalTraversal() [623]
```

GPROF Output – gprof2dot



GPROF - Unaccuracies

HLRS
Thirty Years, One Vision

- Statistical sampling error
- Estimated children time does not take into account the caller
- Recursive functions
- However: Methods and basic block counts are determined accurately

perf

Perf

HLRS
Thirty Years, One Vision

- Included in the Linux kernel
- Allows profiling with performance counters
- Supports two types of events: software (kernel) and hardware (CPU)

Resources:
https://perf.wiki.kernel.org

Perf Usage Overview

HLRS
Thirty Years, One Vision

Statistics for entire program:
perf stat: Run a command and gather performance counter statistics

Detailed profile:
perf record: Run a command and record its profile into perf.data
perf report: Read perf.data and display the profile

Helper:
perf list: List all symbolic event types

perf stat - options

HLRS
Thirty Years, One Vision

Options	
-d -d -d	Display more details
-r=<n>	Repeat command <n> times
-e	Specify PMU* event

perf stat – example output for "-r 5 --table -d -d"

```
Performance counter stats for './fib.orig' (5 runs):

          2,934.37 msec task-clock                #    0.999 CPUs utilized            ( +-  5.36% )
               115      context-switches          #    0.039 K/sec                    ( +- 92.60% )
                 0      cpu-migrations            #    0.000 K/sec                    ( +-100.00% )
                56      page-faults               #    0.019 K/sec                    ( +-  0.88% )
     8,975,453,670      cycles                    #    3.059 GHz                      ( +-  0.44% )  (30.77%)
    25,256,262,425      instructions              #    2.81  insn per cycle           ( +-  0.14% )  (38.52%)
     6,819,249,630      branches                  # 2323.925 M/sec                    ( +-  0.31% )  (38.58%)
         9,791,568      branch-misses             #    0.14% of all branches          ( +-  1.10% )  (38.63%)
     3,414,107,663      L1-dcache-loads           # 1163.490 M/sec                    ( +-  0.36% )  (38.63%)
           195,939      L1-dcache-load-misses     #    0.01% of all L1-dcache hits    ( +- 29.52% )  (38.55%)
            62,013      LLC-loads                 #    0.021 M/sec                    ( +- 33.23% )  (30.75%)
            40,938      LLC-load-misses           #   66.01% of all LL-cache hits     ( +- 33.93% )  (30.72%)
   <not supported>      L1-icache-loads
           532,315      L1-icache-load-misses                                        ( +- 19.15% )  (30.70%)
     3,400,660,603      dTLB-loads                # 1158.908 M/sec                    ( +-  0.38% )  (30.68%)
             1,522      dTLB-load-misses          #    0.00% of all dTLB cache hits   ( +- 40.48% )  (30.66%)
            17,215      iTLB-loads                #    0.006 M/sec                    ( +- 25.13% )  (30.65%)
             2,610      iTLB-load-misses          #   15.16% of all iTLB cache hits   ( +-  9.43% )  (30.67%)
   <not supported>      L1-dcache-prefetches
   <not supported>

            # Final result:
            2.938 +- 0.159 seconds time elapsed  ( +-  5.42% )
```

Perf - Events

HLRS
Thirty Years, One Vision

- It is possible to measure one or more events per run of the perf tool
- Uses time multiplexing for hardware events if not enougth PMU counters available

```
$ perf list
List of pre-defined events (to be used in -e):
  branch-instructions OR branches                    [Hardware event]
  branch-misses                                      [Hardware event]
  cpu-cycles OR cycles                               [Hardware event]
  instructions                                       [Hardware event]
  ref-cycles                                         [Hardware event]
  alignment-faults                                   [Software event]
  bpf-output                                         [Software event]
  context-switches OR cs                             [Software event]
   ...
  L1-dcache-load-misses                              [Hardware cache event]
  L1-dcache-loads                                    [Hardware cache event]
  L1-dcache-stores                                   [Hardware cache event]
  LLC-load-misses                                    [Hardware cache event]
  LLC-loads                                          [Hardware cache event]
  LLC-store-misses                                   [Hardware cache event]
  LLC-stores                                         [Hardware cache event]
```

Perf – attaching to a running process

HLRS
Thirty Years, One Vision

- Get the PID of the process to attach to

```
ps ax | grep sshd
```

- Attach via

```
perf stat ... -p <PID> sleep <time>
```

perf record - sampling

HLRS
Thirty Years, One Vision

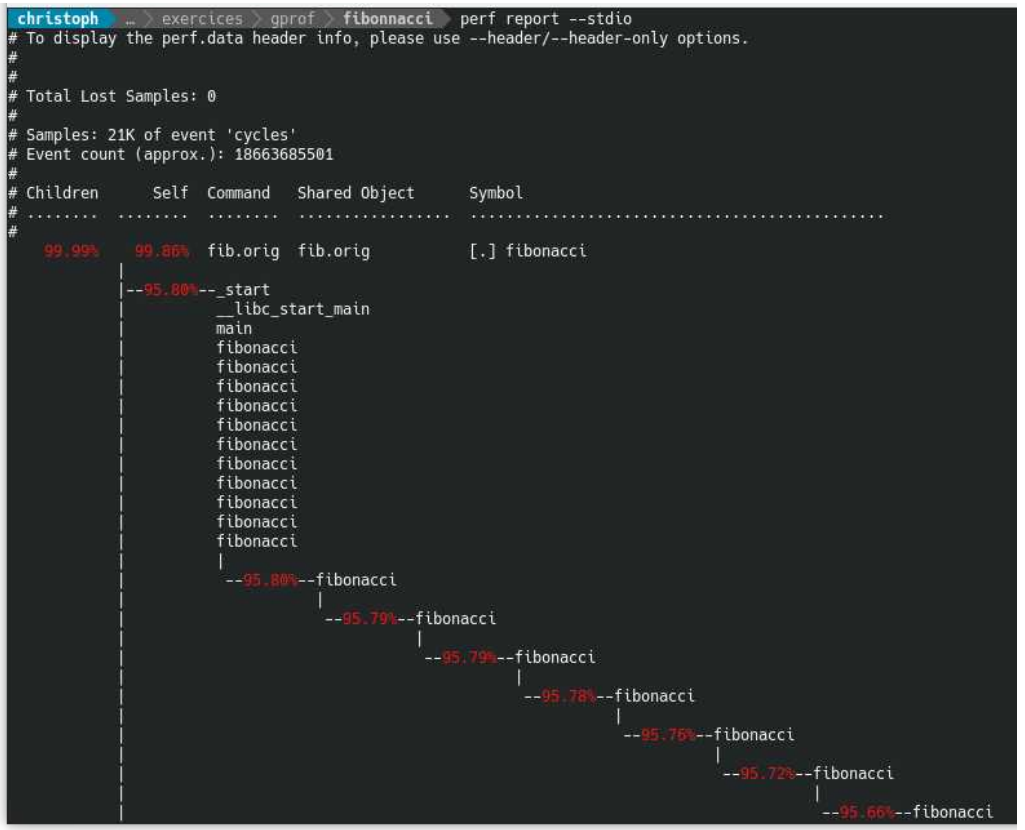
- used to collect profiles on per-thread, per-process and per-cpu basis
- generates an output file called perf.data
- based on event-based sampling
- period is expressed as the number of occurrences of an event, not the number of timer ticks
- Set sampling rate with the -F option if needed

Perf report - analysis

HLRS
Thirty Years, One Vision

- Reads perf.data file and generates a concise execution profile

- Multiple interfaces:
  - --stdio
  - --tui
  - --gtk



mpiP

mpiP – Overview

HLRS
Thirty Years, One Vision

- Lightweight profiling library developed at Lawrence Livermore National Laboratory
- Collects statistical information about MPI calls with relatively low profiling overhead
- Produces a concise text report rather than a detailed event trace
- Typically aggregates information from all MPI ranks when the application finishes

What you get

- MPI time: How much time is spent inside MPI calls
- Hotspots: Which MPI call sites consume the most time
- Call frequency: How often are MPI functions called
- Communication volume: How much data is sent
- Rank variation: Do MPI call times differ significantly between ranks

Limitations

- Statistical summaries, not a detailed time-resolved trace
- Does not directly explain why an MPI call is slow
- Time spent inside MPI is not necessarily time spent transferring data (can include waiting for other processes)
- Does not provide a complete account of non-MPI computation

mpiP – analysis workflow and example

HLRS
Thirty Years, One Vision

Typical workflow

1)Compile application with debug symbols (-g)

2)Link mpiP to your application OR preload its profiling library

3)Run application using the usual MPI launcher.

4)Inspect the generated .mpiP report

MpiP – lulesh2.0 example

HLRS
Thirty Years, One Vision

Example run using the preload mechanism:

```
mpirun -np 8  -x LD_PRELOAD=.../libmpiP.so ./lulesh2.0  …
# for C++ name demangling: c++filt < ./*.mpiP > ./demangled.mpiP
```

```
---------------------------------------
@--- MPI Time (seconds) ---------------
---------------------------------------
Task    AppTime    MPITime     MPI%
   0       5.81       1.17    20.15
   1       5.81        1.5    25.88
   2       5.81       1.57    26.96
   3       5.81       1.67    28.71
   4        5.8       1.77    30.58
   5        5.8       1.72    29.62
   6       5.81       1.72    29.58
   7        5.8       1.81    31.24
   *       46.4       12.9    27.84
```

← Totals

Apptime: wall-clock time from the end of MPI_Init until the beginning of MPI_Finalize
MPI_Time: wall-clock time for all the MPI calls contained within Apptime
MPI%: ratio of this MPI_Time to Apptime

MpiP – lulesh2.0 example

HLRS
Thirty Years, One Vision

```
---------------------------------------------------------------------------
@--- Callsites: 548 ----------------------------------------------------------
---------------------------------------------------------------------------
 ID Lev File/Address        Line Parent_Funct                             MPI_Call
  1   0 lulesh-comm.cc       345 CommRecv(Domain&, int, int, int, int, int, bool, bool)          Irecv
  2   0 lulesh-comm.cc       155 CommRecv(Domain&, int, int, int, int, int, bool, bool)          Irecv
  3   0 lulesh-comm.cc      1757 CommMonoQ(Domain&)                          Wait
  4   0 lulesh-comm.cc       837 CommSend(Domain&, int, int, double& (Domain::**)(int), int, int, int, bool, bool) Isend
  5   0 lulesh-comm.cc       623 CommSend(Domain&, int, int, double& (Domain::**)(int), int, int, int, bool, bool) Isend
...
```

```
---------------------------------------------------------------------------
@--- Aggregate Time (top twenty, descending, milliseconds) ----------------
---------------------------------------------------------------------------
Call                 Site       Time    App%    MPI%      Count    COV
Allreduce             396        932    2.09    8.19         99   0.00
Allreduce             458        829    1.86    7.28         99   0.00
Allreduce             261        801    1.80    7.04         99   0.00
Allreduce             330        776    1.74    6.81         99   0.00
Allreduce             191        640    1.44    5.62         99   0.00
...
```

The Score-P ecosystem

Score-P + CUBE/Scalasca/Vampir

Score-P + CUBE

HLRS
Thirty Years, One Vision

- **Score-P:** instrumentation and performance measurement infrastructure
- **CUBE:** graphical browser for exploring the resulting performance profile
- Supports MPI, OpenMP, and hybrid MPI + OpenMP programs
- Can also support accelerator workflows, including CUDA and HIP, depending on the installation

What you get:
- Hotspots: Which routines consume the most time
- Call paths: Which calling context is responsible for the cost
- MPI overhead: Which communication routines account for significant time
- OpenMP behavior: Where do threads spend time in parallel regions, barriers, and synchronization
- Rank/thread variation: Which processes or threads exhibit different performance

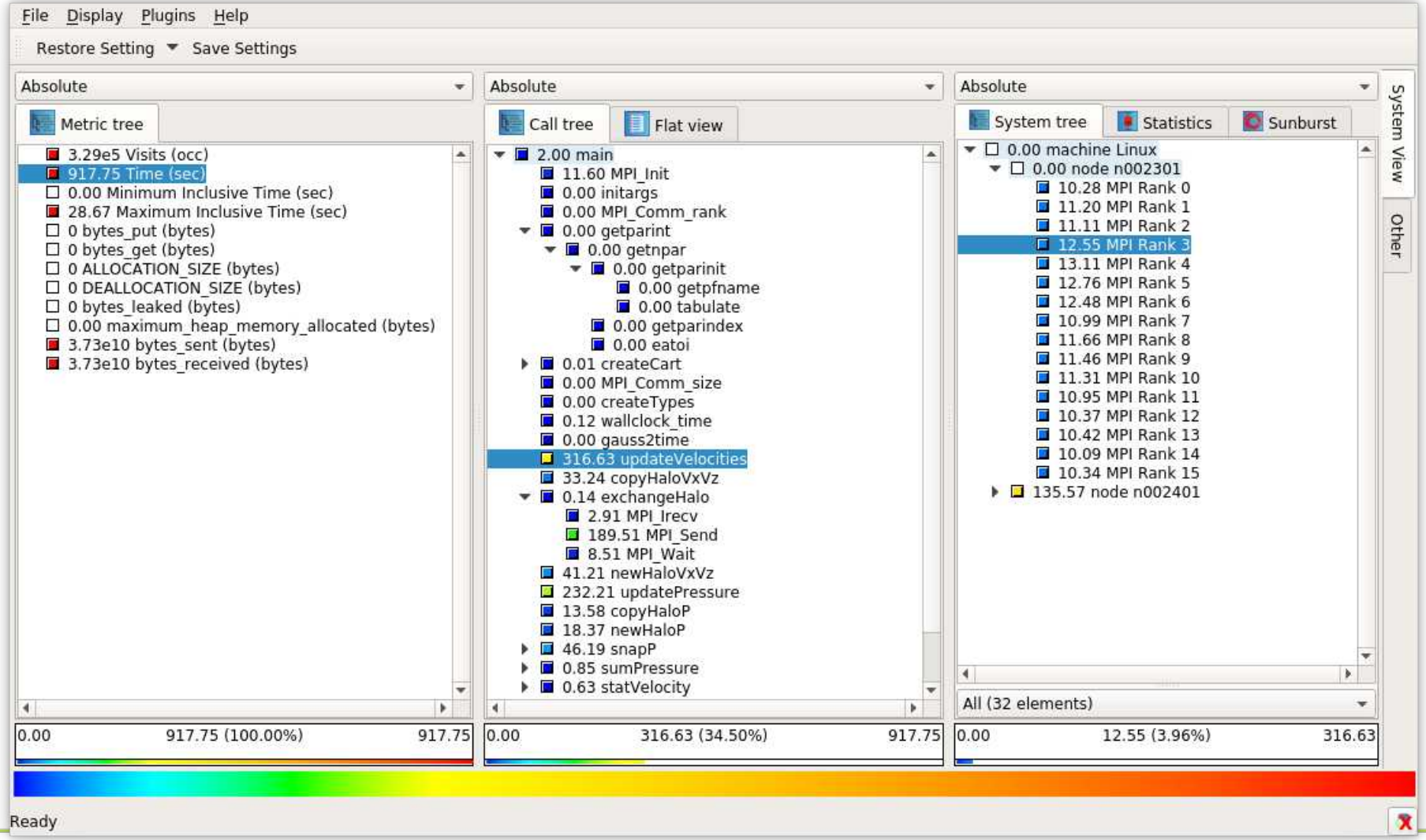
Limitations:
- Instrumentation can introduce very high overheads, especially for C++ applications (Use filters!)
- Profile aggregates measurements; it does not show the detailed chronological sequence of events
- A hotspot identifies where time is spent, but not necessarily the underlying cause
- Detailed timelines require a trace-analysis workflow, for example with Score-P + Vampir or Extrae+Paraver

CUBE

HLRS
Thirty Years, One Vision

- CUBE is a visualizer for profiling data
- Profiling data have to be obtained seperately with the Score-P tracing and profiling tool
- Score-P requires to recompile and link your program. Once done one runs the program for analysis as usual.
- The details of the data collection are controlled via environment variables.
- For more detailed information the profiling and tracing data can be anayzed in a second step with the Scalasca tool, enriching it with extra information. (optional)
- The profilig data can then be viewed with the CUBE visualizer

Scalasca + CUBE



Extrae & Paraver

Extrae + Paraver – Overview

- Developed by the Barcelona Supercomputing Center (BSC)
- **Extrae:** Instruments applications and records timestamped execution events + hardware counter
- **Paraver:** Visualizes and analyzes execution traces
- Supports MPI, OpenMP, hybrid applications, and other programming models

What you get:

- Load imbalance: Differences in execution time across ranks and threads
- Communication: MPI message activity, communication patterns, and waiting times
- Synchronization: Barriers, locks, and idle periods
- Execution timeline: When and where performance bottlenecks occur

Limitations:

- Traces can become very large for long-running or highly parallel applications
- Instrumentation and tracing introduce overhead
- Requires more storage and analysis effort than lightweight profiling
- Limited in scalability (~100-1000 processes*threads)

Paraver usage

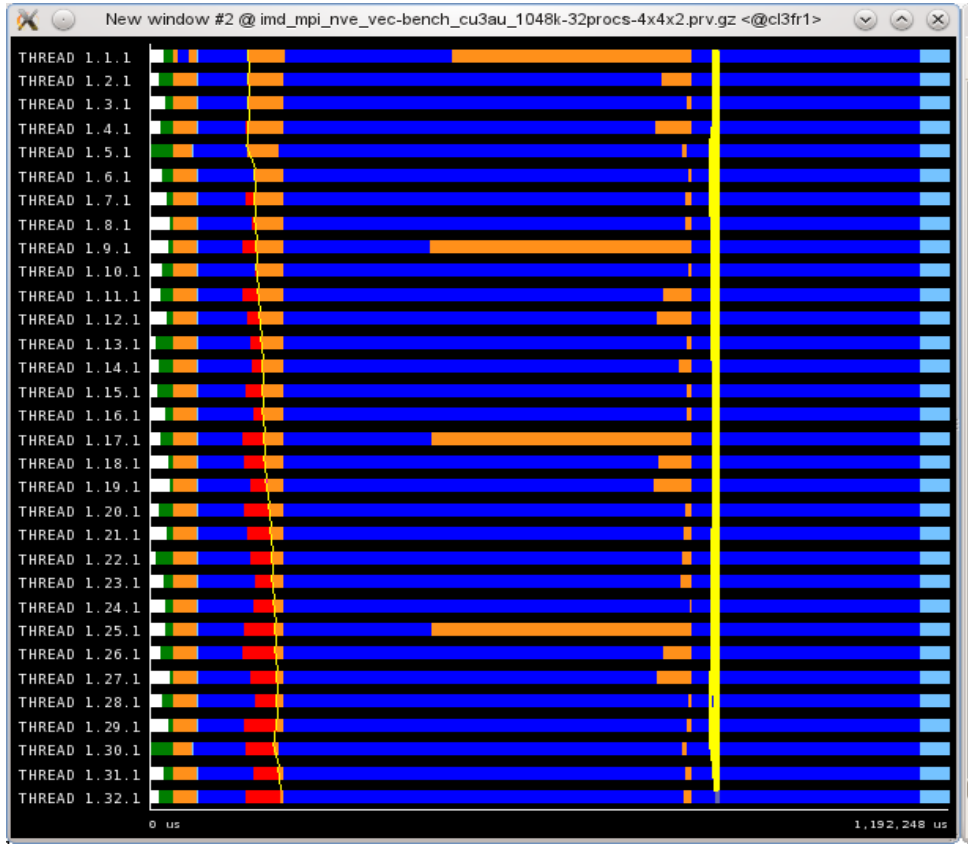
HLRS
Thirty Years, One Vision

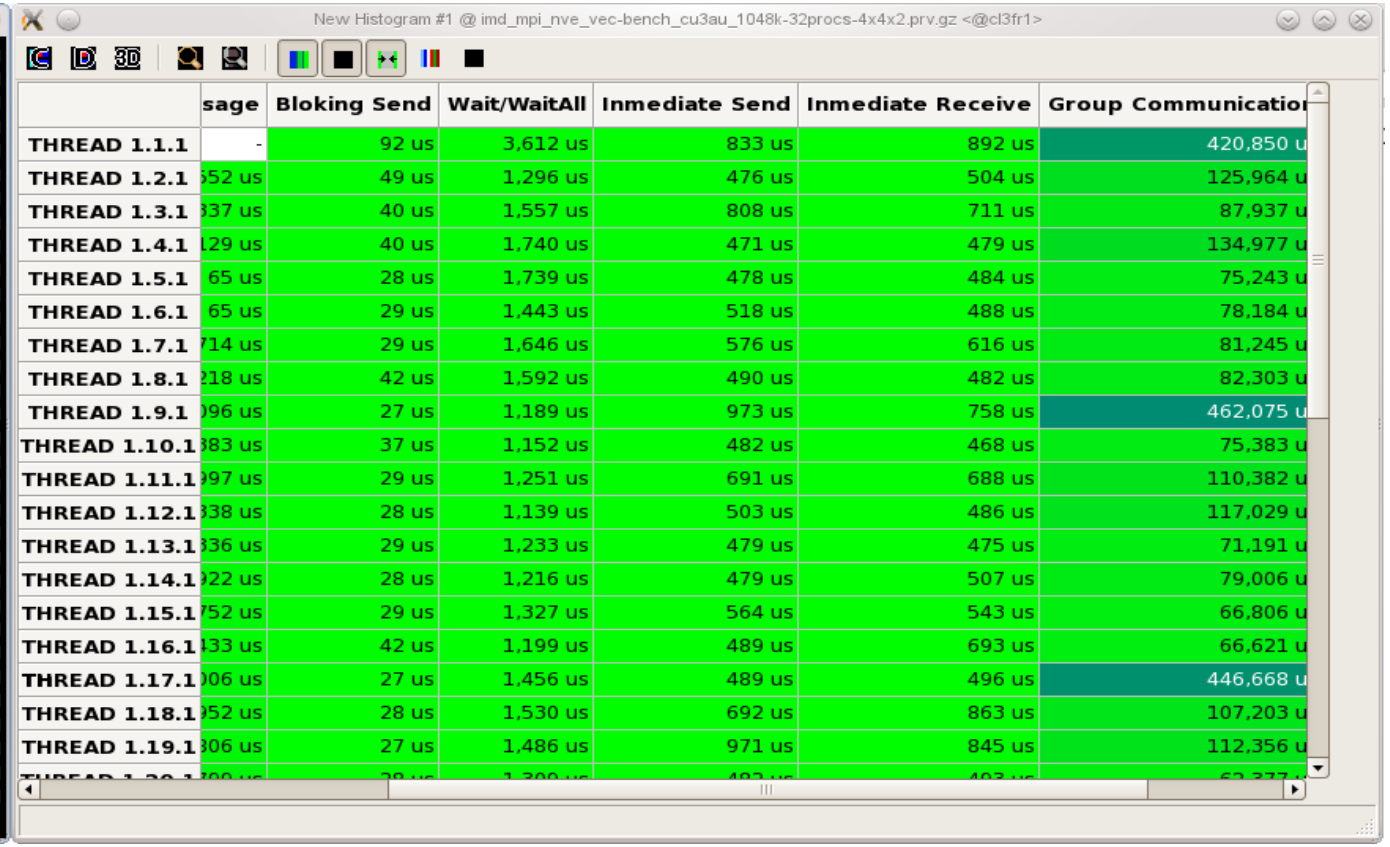
- Paraver is a very flexible analysis tool and visualizer for tracing data
- Tracing data have to be collected seperately using the extrae library
- Tracing with Extrace can be controlled with a configuration file
- It will generate a set of files with the tracing data (*.prv, *.row) that can then be viewed in Paraver

Typical workflow

1)Set up an Extrae configuration file
2)Run the application and collect execution traces (preload mechanism)
3)Generate and merge trace files if needed
4)Open the combined trace in Paraver.
5)Inspect various metrics using statistics views and timelines across processes and threads to identify and study bottlenecks in detail

Paraver


New window #2 @ imd_mpi_nve_vec-bench_cu3au_1048k-32procs-4x4x2.prv.gz <@cl3fr1>
THREAD 1.1.1
THREAD 1.2.1
THREAD 1.3.1
THREAD 1.4.1
THREAD 1.5.1
THREAD 1.6.1
THREAD 1.7.1
THREAD 1.8.1
THREAD 1.9.1
THREAD 1.10.1
THREAD 1.11.1
THREAD 1.12.1
THREAD 1.13.1
THREAD 1.14.1
THREAD 1.15.1
THREAD 1.16.1
THREAD 1.17.1
THREAD 1.18.1
THREAD 1.19.1
THREAD 1.20.1
THREAD 1.21.1
THREAD 1.22.1
THREAD 1.23.1
THREAD 1.24.1
THREAD 1.25.1
THREAD 1.26.1
THREAD 1.27.1
THREAD 1.28.1
THREAD 1.29.1
THREAD 1.30.1
THREAD 1.31.1
THREAD 1.32.1
0 us
1,192,248 us


New Histogram #1 @ imd_mpi_nve_vec-bench_cu3au_1048k-32procs-4x4x2.prv.gz <@cl3fr1>
sage
Bloking Send
Wait/WaitAll
Inmediate Send
Inmediate Receive
Group Communicatio
THREAD 1.1.1 - 92 us 3,612 us 833 us 892 us 420,850 u
THREAD 1.2.1 552 us 49 us 1,296 us 476 us 504 us 125,964 u
THREAD 1.3.1 337 us 40 us 1,557 us 808 us 711 us 87,937 u
THREAD 1.4.1 129 us 40 us 1,740 us 471 us 479 us 134,977 u
THREAD 1.5.1 65 us 28 us 1,739 us 478 us 484 us 75,243 u
THREAD 1.6.1 65 us 29 us 1,443 us 518 us 488 us 78,184 u
THREAD 1.7.1 714 us 29 us 1,646 us 576 us 616 us 81,245 u
THREAD 1.8.1 218 us 42 us 1,592 us 490 us 482 us 82,303 u
THREAD 1.9.1 096 us 27 us 1,189 us 973 us 758 us 462,075 u
THREAD 1.10.1 383 us 37 us 1,152 us 482 us 468 us 75,383 u
THREAD 1.11.1 997 us 29 us 1,251 us 691 us 688 us 110,382 u
THREAD 1.12.1 338 us 28 us 1,139 us 503 us 486 us 117,029 u
THREAD 1.13.1 336 us 29 us 1,233 us 479 us 475 us 71,191 u
THREAD 1.14.1 922 us 28 us 1,216 us 479 us 507 us 79,006 u
THREAD 1.15.1 752 us 29 us 1,327 us 564 us 543 us 66,806 u
THREAD 1.16.1 433 us 42 us 1,199 us 489 us 693 us 66,621 u
THREAD 1.17.1 006 us 27 us 1,456 us 489 us 496 us 446,668 u
THREAD 1.18.1 952 us 28 us 1,530 us 692 us 863 us 107,203 u
THREAD 1.19.1 806 us 27 us 1,486 us 971 us 845 us 112,356 u

Performance-Aspects and -Analysis for MPI & OpenMP applications

HLRS

Thirty Years, One Vision



Christoph Niethammer <niethammer@hlrs.de>